

# Imagination PowerVR PVRTune 完整使用指南與 Performance Counter 定義解析

本文件詳細彙整 Imagination Technologies PowerVR GPU 專屬效能剖析工具 PVRTune 的系統架構、連線與操作流程、全套硬體效能計數器（Performance Counters）的定義與計算公式，以及基於 TBDR 架構的效能瓶頸分析（Bottleneck Analysis）實戰指南。

## 0. PVRTune 系統架構與核心概念

PVRTune 是針對 PowerVR 架構 GPU 打造的低開銷（Low-Overhead）即時硬體效能分析工具。它直接讀取 GPU 硬體暫存器與驅動程式內部事件，以時間軸（Timeline）與即時波形呈現繪圖管線的各階段負載。



### 0.1 核心三大組成元件

1. PVRTune GUI：主機端視覺化應用程式，支援即時連線監控（Live Profiling）與離線分析檔案載入（.pvrtune / .pvrperf）。
2. PVRPerfServer：目標裝置端背景常駐程式（Server），直接向 PowerVR 核心驅動查詢硬體效能計數器與任務排程事件，並透過網路串流傳輸至 GUI。
3. PVRScope Library：C/C++ 函式庫，開發者可將其編譯至遊戲/應用程式內：
  - 在應用程式內部即時讀取 GPU 硬體計數器，進行動態畫質/效能調整。
  - 向 PVRTune 時間軸發送自訂 Mark 與時間標記（Custom Counters / Breadcrumbs）。

## 1. PVRTune 連線與使用流程

### 1.1 Android 裝置連線步驟

- 步驟 1: 推送 PVRPerfServer 至目標裝置

adb push PVRPerfServer /data/local/tmp/  
adb shell chmod 777 /data/local/tmp/PVRPerfServer
- 步驟 2: 啟動背景伺服器

adb shell /data/local/tmp/PVRPerfServer &

#### 步驟 3: 設定 USB Port 轉發 (Port Forwarding)

adb forward tcp:6520 tcp:6520

#### 步驟 4: 開啟 PVRTune GUI 並連線

點擊 Connect -> 輸入 127.0.0.1:6520 -> 開始 Profiling

### 1.2 Linux / 嵌入式平台連線步驟

1. 確保目標裝置網路暢通，於目標板執行：`./PVRPerfServer &`。
2. 主機端開啟 PVRTune GUI，點擊 Connection Window。
3. 輸入目標板 IP 位址（例如 **192.168.1.100**）與連接埠（預設 **6520**），點擊 Connect。

### 1.3 操作技巧與資料收集

- 即時監控 (Live Monitoring)：點擊 Start Recording，即時觀察各核心負載曲線。
- 儲存分析紀錄 (Save Session)：錄製完成後存為 **.pvr\_tune** 檔，供離線比對與效能回溯。
- 自訂視圖 (Custom Layouts)：將常用的計數器（如 GPU Core Load、HSR Efficiency、USC Tasks）釘選至最上方監控面板。

## 2. PowerVR TBDR 架構與時間軸對照

PowerVR 架構採用 Tile-Based Deferred Rendering (TBDR)，將渲染任務解耦為兩大非同步平行階段：

#### PowerVR TBDR 雙階段硬體時間軸 (Timeline)

↓ Frame N-1 Frame N Frame N+1

#### Tiler (TA) Phase: 幾何變換、裁剪、分箱至 32x32 Tiles

[TA: Frame N-1] ---> [TA: Frame N] ---> [TA: Frame N+1]

↓ \ \ \ (Tile Lists) \ (Tile Lists)

#### Renderer (3D) Phase: ISP 隱藏面剔除 (HSR) -> USC 像素著色 -> PBE 寫回

[3D: Frame N-1] ---> [3D: Frame N]

#### 1. TA (Tiling Acceleration / Geometry Phase)：

- 執行 Vertex Shader、坐標變換、投影、裁剪剔除 (Clip/Cull)，將圖元依照空間邊界寫入 Parameter Buffer，並為每個 32x32 Tile 建立圖元參照清單。

#### 2. 3D (Rendering / Pixel Phase)：

- 逐 Tile 載入圖元參照，透過 ISP 進行完美的 HSR (Hidden Surface Removal)，只對最終可見的像素執行 Fragment Shader (USC)，徹底消弭 Overdraw 造成的算力與頻寬浪費。

#### 3. 並行重疊 (Pipelined Overlap)：

- 當 3D Core 在渲染第  $N-1$  影格時，Tiler 核心同時在處理第  $N$  影格的幾何，兩者在硬體上平行運作。

## 3. Performance Counters 核心分類與定義

PVRTune 提供的計數器可分為七大類別：

### 3.1 核心總體負載計數器 (Core & Top-Level Loads)

| 計數器名稱 (Counter Name) | 單位 | 定義與說明 | 理想目標與警戒門檻 |
|----------------------|----|-------|-----------|
|----------------------|----|-------|-----------|

|                                      |   |                                             |                                |
|--------------------------------------|---|---------------------------------------------|--------------------------------|
| <b>**GPU Core Load**</b>             | % | GPU 任何硬體單元處於工作狀態的時間百分比。綜合反映 GPU 整體繁忙程度。     | < 85%（超過 90% 即表示 GPU 飽和）       |
| <b>**Tiler Active (TA Load)**</b>    | % | Tiling/幾何處理單元處於活動狀態的時間比例。負責頂點著色、裁剪與分箱。      | > 70% 表示頂點/多邊形過多或幾何計算過重        |
| <b>**Renderer Active (3D Load)**</b> | % | 像素著色與光柵化單元處於活動狀態的時間比例。負責 ISP、USC 片段著色與 PBE。 | 主流遊戲瓶頸多在此處，應配合 USC/ISP 指標分析    |
| <b>**2D Active / TDM Active**</b>    | % | 2D 傳輸、Blit、色彩清除（Clear）或格式轉換引擎活動比例。          | 正常應較低；過高代表頻繁的 FBO 複製或未對齊的 Blit |
| <b>**Compute Active**</b>            | % | 計算著色器（Compute Shader / OpenCL）佔用核心的時間比例。    | 依計算任務需求而定                      |

### 3.2 幾何與 Tiler (TA) 計數器

| 計數器名稱 (Counter Name)                  | 單位 | 定義與說明                                                  | 分析意義與優化方向                                 |
|---------------------------------------|----|--------------------------------------------------------|-------------------------------------------|
| <b>**Vertices Input**</b>             | 數量 | 傳入 GPU 幾何管線的頂點總數。                                      | 衡量場景原始複雜度；可透過 LOD 或網格簡化降低                 |
| <b>**Vertices Shaded**</b>            | 數量 | 實際執行 Vertex Shader 的頂點數量（受 Post-Transform Cache 命中影響）。 | 若與 Vertices Input 接近，表示 Index Cache 命中率不佳 |
| <b>**Triangles Input**</b>            | 數量 | 提交給硬體的原始三角形數量。                                         | 評估 Draw Call 幾何規模                         |
| <b>**Triangles Clipped / Culled**</b> | 數量 | 在視錐體裁剪（Clip）與背面剔除（Cull）階段被丟棄的三角形數。                     | 若 Culled 比例極高，應考慮在 CPU 端實施粗粒度遮擋剔除         |
| <b>**Parameter Buffer Memory**</b>    | MB | Tiler 寫入 Parameter Buffer 的幾何與控制參數總記憶體量。               | 若超出限制會觸發 SPM（Scene Parameter Memory）溢位    |

### 3.3 ISP 與 HSR (隱藏面剔除) 計數器

| 計數器名稱 (Counter Name)                   | 單位 | 定義與說明                                                                                   | 分析意義與優化方向                                                                                                      |
|----------------------------------------|----|-----------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|
| <b>**HSR Efficiency**</b>              | %  | 隱藏面剔除效率： $\frac{\text{被 ISP 剔除的被遮擋像素數}}{\text{覆蓋測試通過的總像素數}} \times 100\%$ 。衡量避開無效著色的比例。 | 正常複雜場景應在 60%~95%；極低可能因全屏透明混合或 Discard 破壞 HSR                                                                   |
| <b>**ISP Tiles Processed**</b>         | 數量 | 單影格內完成處理的 32×32 Tile 總數。                                                                | 標準 1080p 畫面約為 $\lceil \frac{1920}{32} \rceil \times \lceil \frac{1080}{32} \rceil = 60 \times 34 = 2040$ Tiles |
| <b>**Depth Tested Fragments**</b>      | 數量 | 送入 ISP 進行深度比對的片段總數。                                                                     | 評估場景深度測試壓力                                                                                                     |
| <b>**Depth Rejected Fragments**</b>    | 數量 | 經深度測試或早期 HSR 判定不通過而被剔除的片段數。                                                             | 反映幾何遮擋深度測試的成效                                                                                                  |
| <b>**Opaque vs Translucent Ratio**</b> | 比值 | 不透明物體與半透明混合（Alpha Blending）物體的處理比例。                                                     | 半透明物件無法完全被 Early HSR 剔除，過多會大幅增加像素處理負擔                                                                          |

### 3.4 USC (著色核心) 與 ALU 計數器

| 計數器名稱 (Counter Name) | 單位 | 定義與說明 | 分析意義與優化方向 |
|----------------------|----|-------|-----------|
|----------------------|----|-------|-----------|

|                                     |    |                                                         |                                                 |
|-------------------------------------|----|---------------------------------------------------------|-------------------------------------------------|
| <b>**Shader Processing Load**</b>   | %  | USC 著色器核心執行指令的平均時間比例。                                   | 超過 80% 代表著色器算力受限（Shader Bound）                  |
| <b>**Processing Load: Pixel**</b>   | %  | USC 執行 Fragment / Pixel Shader 的負載比例。                   | 最常見的算力瓶頸                                        |
| <b>**Processing Load: Vertex**</b>  | %  | USC 執行 Vertex Shader 的負載比例。                             | 頂點計算或骨骼動畫權重計算負擔                                 |
| <b>**Processing Load: Compute**</b> | %  | USC 執行 Compute Shader 的負載比例。                            | 通用計算負載                                          |
| <b>**ALU F32 Load**</b>             | %  | USC 內部 32 位元高精度（ <b>highp</b> ）浮點運算單元的負載。               | 過高應考慮降轉為 <b>mediump</b> 16-bit 運算               |
| <b>**ALU F16 Load**</b>             | %  | USC 內部 16 位元半精度（ <b>mediump</b> ）浮點運算單元的負載（通常具 2X 輸送量）。 | 提高 F16 佔比可倍增著色器 ALU 吞吐量並省電                      |
| <b>**USC Tasks in Flight**</b>      | 數量 | 同時在 USC 中處於執行或就緒狀態的 Task 數量（並行度）。                       | 過低表示暫存器佔用過高（Register Pressure）限制了佔用率（Occupancy） |

### 3.5 TPU (紋理處理單元) 計數器

| 計數器名稱 (Counter Name)                        | 單位 | 定義與說明                             | 分析意義與優化方向                              |
|---------------------------------------------|----|-----------------------------------|----------------------------------------|
| <b>**Texture Processing Load**</b>          | %  | TPU 紋理採樣單元處於忙碌狀態的時間比例。            | > 75% 代表紋理採樣成為瓶頸（Texture Bound）        |
| <b>**Texel Fetches**</b>                    | 數量 | TPU 自快取或記憶體讀取 Texel 的總次數。         | 評估紋理讀取頻繁度                              |
| <b>**Texture Filter: Bilinear Cycles**</b>  | 週期 | 執行 4-tap 雙線性過濾消耗的時脈週期。            | 衡量基本紋理過濾開銷                             |
| <b>**Texture Filter: Trilinear Cycles**</b> | 週期 | 執行 8-tap 三線性過濾與 Mipmap 插值消耗的時脈週期。 | 若過高可評估是否需全程啟用三線性過濾                     |
| <b>**Texture Cache Hit Rate**</b>           | %  | TCU (Texture Cache Unit) 命中率。     | 應維持在 > 85%；過低通常因未開啟 Mipmap 或 UV 座標跨度過大 |

### 3.6 記憶體階層與頻寬計數器 (Memory & Bandwidth)

| 計數器名稱 (Counter Name)                     | 單位   | 定義與說明                                    | 分析意義與優化方向                              |
|------------------------------------------|------|------------------------------------------|----------------------------------------|
| <b>**GPU Memory Interface Load**</b>     | %    | GPU 外部記憶體匯流排 (System Bus / DRAM) 的頻寬利用率。 | 最關鍵指標之一。> 75% 表示頻寬瓶頸（Bandwidth Bound）  |
| <b>**System Memory Read Bandwidth**</b>  | MB/s | GPU 自系統 DRAM 讀取的即時頻寬吞吐量。                 | 包含紋理載入、頂點讀取、Tile 參數讀取                  |
| <b>**System Memory Write Bandwidth**</b> | MB/s | GPU 寫入系統 DRAM 的即時頻寬吞吐量。                  | 包含 Framebuffer 寫回、Parameter Buffer 溢位等 |
| <b>**SLC Read / Write Throughput**</b>   | MB/s | System Level Cache（系統級共用快取）的讀寫吞吐量。       | 衡量快取層對外存頻寬的過濾成效                        |

### 3.7 驅動程式與 API 呼叫計數器

| 計數器名稱 (Counter Name)             | 單位 | 定義與說明                                              | 分析意義與優化方向                                  |
|----------------------------------|----|----------------------------------------------------|--------------------------------------------|
| <b>**API Active Time**</b>       | %  | CPU 在圖形驅動 (OpenGL ES / Vulkan) 內部的耗時比例。            | 過高表示 CPU 端驅動開銷過重 (CPU Overhead)            |
| <b>**Draw Calls / Frame**</b>    | 次數 | 每影格發出的繪圖指令呼叫次數 ( <b>glDrawArrays, vkCmdDraw</b> )。 | 建議行動端控制在 500~1500 次以內，過多應進行批次合併 (Batching) |
| <b>**State Changes / Frame**</b> | 次數 | 管線狀態 (FBO、Shader、Texture、Blend) 切換次數。              | 頻繁切換狀態會中斷硬體快取並增加 CPU 負擔                    |

## 4. 效能瓶頸分析決策樹 (Bottleneck Analysis Decision Tree)

在 PVRTune 中，可遵循以下標準決策樹迅速定位效能瓶頸：

**步驟 1: 檢查總體時間與 GPU Core Load**  
IF GPU Core Load < 70% 且 FPS 低於目標:  
--> **【CPU Bound】**: CPU 提交過慢，檢查 API Active Time 與 Draw Calls  
ELSE IF GPU Core Load >= 85%:  
--> 進入步驟 2 (GPU 內部瓶頸判定)

**步驟 2: 比較 Tiler Active (TA) 與 Renderer Active (3D)**  
IF Tiler Active > 75% 且 Tiler Active > Renderer Active:  
--> **【幾何/頂點瓶頸 (Geometry Bound)】**: 頂點數過多或 Vertex Shader 過重  
ELSE IF Renderer Active > 75%:  
--> 進入步驟 3 (著色/紋理/頻寬細分)

**步驟 3: 診斷 3D 渲染階段瓶頸**  
3a. 檢查 GPU Memory Interface Load:  
IF > 75%: --> **【記憶體頻寬受限 (Memory Bandwidth Bound)】**  
對策: 壓縮紋理 (PVRTC/ASTC)、縮減解析度、啟用 Mipmap  
3b. 檢查 Texture Processing Load:  
IF > 75% 且 TCU Hit Rate < 80%:  
--> **【紋理採樣受限 (Texture Bound)】**  
對策: 檢查 Mipmap 是否遺漏、優化 UV 取樣連續性  
3c. 檢查 Shader Processing Load (Pixel):  
IF > 75% 且 ALU F32 佔比極高:  
--> **【著色器算力受限 (Shader ALU Bound)】**  
對策: 將 **highp** 改為 **mediump**、簡化數學運算  
3d. 檢查 HSR Efficiency:  
IF < 50% 且場景複雜:  
--> **【Overdraw / 隱藏面剔除失效】**  
對策: 減少 Alpha Test (Discard)、控制半透明混合排序

## 5. 常見瓶頸與 PowerVR 專屬優化對策

### 5.1 記憶體頻寬瓶頸 (Bandwidth Bound)

- 現象：**GPU Memory Interface Load** 居高不下，幀率波動劇烈。
- 優化對策：
  1. 全面採用硬體紋理壓縮：使用 PVRTC 或 ASTC 格式，避免使用無壓縮的 RGBA8888 貼圖。
  2. 務必生成並啟用 Mipmaps：未開啟 Mipmap 會導致遠景紋理採樣快取命中率 (TCU Hit Rate) 崩潰，急遽拉高 DRAM 頻寬。
  3. 頂點屬性緊湊化 (Interleaved & Packed Attributes)：使用 16-bit 定點數或半精度浮點數 (**HALF\_FLOAT**) 封裝頂點法線與 UV 坐標。

5.2 著色器算力瓶頸 (Shader ALU Bound)

- 現象：**Shader Processing Load (Pixel)** 接近 100%，**ALU F32 Load** 比例極高。
- 優化對策：
  1. \*\*優先使用 **mediump** (F16) \*\*：PowerVR USC 具備雙發 (Dual-issue) F16 ALU，使用 **mediump** 可使算力吞吐量翻倍並大幅減少暫存器佔用。
  2. 避免動態分支與昂貴數學運算：減少在 Fragment Shader 內計算 **pow()**, **sin()**, **exp()**，改用預先烘焙 (Pre-computed) 紋理查表。
  3. 降低暫存器壓力 (Register Pressure)：過多的局部變數會降低 USC 并行度 (Occupancy)，導致延遲無法被掩蓋。

5.3 破壞 TBDR HSR 機制 (HSR Inefficiency & Overdraw)

- 現象：**HSR Efficiency** 異常低落，Renderers 耗時過長。
- 優化對策：
  1. \*\*慎用 **discard** (Alpha Test) 與 **gl\_FragDepth**\*\*：在 Fragment Shader 中呼叫 **discard** 或動態寫入深度會強制硬體關閉 Early HSR，退化為延遲寫入。
  2. 區分不透明 (Opaque) 與半透明 (Alpha Blend) 渲染 Pass：不透明物體先行繪製，充分利用 HSR 剔除被遮擋物體；半透明物體隨後繪製。
  3. 避免冗餘清除 (Redundant Clears) 與保留 Framebuffer：每影格起始呼叫 **glClear**，讓 Tiler 與 ISP 能夠在 Tile 記憶體內直接初始化，避免自 DRAM 載入舊圖 (Load Op: Clear vs Load)。

6. PVRTune GUI 介面解析

1. Timeline View (頂部時間軸視圖)

顯示 CPU 執行緒、驅動程式提交區間、TA 與 3D 任務在時間軸上的重疊塊。  
點擊任務塊可查看該 RenderPass / DrawCall 的耗時與關聯 API 呼叫。

↓ 2. Counter Graph View (即時曲線圖) | 以多色波形即時繪製選定計數器 (如 GPU Load, ALU F16, Mem Bandwidth)。 | 支援放大 (Zoom in/out)、拖曳與標記特定影格範圍。 |

3. Counter Properties & Statistical Summary (底部屬性與統計面板)

顯示所選時間區間內各計數器的 平均值 (Avg)、最大值 (Max) 與 總和 (Sum)。  
提供計數器的文字定義說明與正常範圍提示。

7. 總結

| 分析維度    | 關鍵監控指標                                      | 核心目標                                        |
|---------|---------------------------------------------|---------------------------------------------|
| 整體效能    | GPU Core Load, FPS                          | 保持平穩在 60/120 FPS，負載維持在 70%~85% 安全水位         |
| 管線平衡    | Tiler Active vs Renderers Active            | 兩者時間維持平衡，避免單一核心長時間等待                        |
| 幾何效率    | Vertices Input, Triangles Culled            | 透過 LOD 與視錐裁剪減少進入 Tiler 的無效頂點                |
| 光柵與 HSR | HSR Efficiency, Depth Rejected Fragments    | 維持 HSR 效率 > 80%，減少 <b>discard</b> 與無效混合     |
| 著色器效率   | ALU F16 Load %, Shader Processing Load      | 最大化 <b>mediump</b> 佔比，最小化 <b>highp</b> 算力開銷 |
| 記憶體頻寬   | GPU Memory Interface Load, System Bandwidth | 全面使用 PVRTC/ASTC 與 Mipmaps，壓低外存頻寬            |